Week 6 Assignment
10-03-2026

Database Performance Tuning and Normalization

Problem Statement

Study and understand the concepts of **Database Performance Tuning** and **Database Normalization**.
Explain how performance tuning improves database efficiency and describe the different levels of normalization including:

- First Normal Form (1NF)
- Second Normal Form (2NF)
- Boyce-Codd Normal Form (BCNF)
- Fourth Normal Form (4NF)
- Fifth Normal Form (5NF)

Provide clear explanations with examples to demonstrate how normalization reduces redundancy and improves database design.

1. Performance Tuning

Definition

Database performance tuning is the process of optimizing database performance to ensure queries run efficiently and system resources are used effectively. The goal is to reduce response time and improve overall database performance.

Importance of Performance Tuning

- Improves query execution speed
- Reduces system resource usage
- Enhances scalability
- Improves user experience
- Ensures efficient data retrieval

Techniques Used in Performance Tuning

1. Indexing

Indexes help speed up data retrieval by allowing the database to locate records quickly without scanning the entire table.

Example:

```
CREATE INDEX idx_employee_name
ON Employee(name);
```

2. Query Optimization

Writing efficient SQL queries helps reduce execution time.

Example:
Using specific columns instead of selecting all columns.

```
SELECT name, salary FROM Employee;
```

instead of

```
SELECT * FROM Employee;
```

3. Proper Database Design

Efficient schema design reduces redundancy and improves performance.

4. Using Joins Efficiently

Proper joins help retrieve related data efficiently.

5. Avoiding Unnecessary Data Retrieval

Fetching only required records improves performance.

Example:

```
SELECT * FROM Employee
WHERE salary > 50000;
```

---

2. Database Normalization

Definition

Normalization is a database design technique used to organize data in a database to reduce redundancy and improve data integrity. It divides large tables into smaller related tables and defines relationships between them.

First Normal Form (1NF)

Definition

A table is in **First Normal Form** if:

- All columns contain atomic (indivisible) values
- Each record is unique
- No repeating groups are allowed

Example

Not in 1NF

Student_ID	Name	Subjects
1	Rahul	Math, Science
2	Anita	English, History

In 1NF

Student_ID	Name	Subject
1	Rahul	Math
1	Rahul	Science
2	Anita	English
2	Anita	History

Second Normal Form (2NF)

Definition

A table is in **Second Normal Form** if:

- It is already in **1NF**
- All non-key attributes depend on the **entire primary key**

Example

Not in 2NF

Order_ID Product_ID Product_Name Quantity

Here **Product_Name depends only on Product_ID**, not on the full key.

In 2NF

Split into two tables:

Products Table

| Product_ID | Product_Name |

Orders Table

| Order_ID | Product_ID | Quantity |

Boyce-Codd Normal Form (BCNF)

Definition

A table is in **BCNF** if:

- It is already in **3NF**
- Every determinant is a candidate key

Example

| Course | Instructor | Room |

If an instructor teaches only one course, then **Instructor determines Course**, causing dependency issues.

Solution: Split into two tables.

Instructor Table

| Instructor | Course |

Course Room Table

| Course | Room |

Fourth Normal Form (4NF)

Definition

A table is in **Fourth Normal Form** if:

- It is already in **BCNF**
- It has **no multi-valued dependencies**

Example

| Student | Hobby | Language |

A student can have multiple hobbies and languages.

Solution

Split into two tables:

Student_Hobby

| Student | Hobby |

Student_Language

| Student | Language |

Fifth Normal Form (5NF)

Definition

A table is in **Fifth Normal Form** if:

- It is already in **4NF**
- It cannot be further decomposed without losing information
- It removes **join dependencies**

Example

| Supplier | Product | Project |

This table may be decomposed into:

Supplier_Product
Supplier_Project
Product_Project

These tables can be joined back to reconstruct the original table.

Benefits of Normalization

- Eliminates data redundancy
- Improves data consistency
- Reduces storage space
- Improves database structure
- Simplifies database maintenance

11-03-2026

Problem Statement

A retail company wants to analyze its **sales performance, customer behavior, and product demand** to improve profitability.

Management wants answers to the following questions:

- Which **products generate the highest sales**
- Which **categories perform the best**
- Which **regions contribute the most revenue**
- How **sales change over time**
- Which products have **low performance**

Build an **interactive Power BI dashboard** that helps management track sales trends and make better business decisions.

Dataset Fields

Retail datasets usually include fields like:

```
Order_ID
Order_Date
Customer_ID
Product_Name
Category
Region
Quantity
Unit_Price
Sales
Profit
```

Step 1 — Import Dataset

Open **Microsoft Power BI Desktop**

1. Click **Get Data**
2. Choose **CSV / Excel**
3. Load the retail dataset.

Step 2 — Create Important Measures (KPIs)

Go to **Modeling → New Measure**

Total Sales

```
Total Sales = SUM(Retail[Sales])
```

Total Profit

```
Total Profit = SUM(Retail[Profit])
```

Total Orders

```
Total Orders = DISTINCTCOUNT(Retail[Order_ID])
```

Profit Margin

```
Profit Margin = DIVIDE([Total Profit],[Total Sales])
```

Total Quantity Sold

```
Total Quantity = SUM(Retail[Quantity])
```

Step 3 — KPI Cards (Top Section)

Create **Card visuals**.

Add KPIs:

```
Total Sales
Total Profit
Total Orders
Profit Margin
Total Quantity Sold
```

Example layout:

```
-------------------------------------------------
Sales | Profit | Orders | Margin | Quantity
-------------------------------------------------
```

This gives **instant business performance insight**.

Step 4 — Sales Trend Over Time

Add **Line Chart**

Fields:

```
X-Axis → Order_Date
Values → Total Sales
```

Insight:

Shows sales growth and seasonal trends.

Step 5 — Sales by Category

Add **Bar Chart**

```
Axis → Category
Values → Total Sales
```

Insight:

Identify best performing product categories.

Step 6 — Sales by Region

Add **Column Chart**

```
Axis → Region
Values → Total Sales
```

Insight:

Shows geographic sales performance.

Step 7 — Top Products

Add **Bar Chart**

```
Axis → Product_Name
Values → Total Sales
```

Sort **descending** to show **Top 10 products**.

Step 8 — Category Profitability

Add **Stacked Column Chart**

```
Axis → Category
Values → Sales and Profit
```

Insight:

Shows which categories generate revenue but low profit.

Step 9 — Product Sales Distribution

Add **Treemap**

```
Group → Product_Name
Values → Total Sales
```

Insight:

Identify products contributing most revenue.

Step 10 — Add Interactive Filters

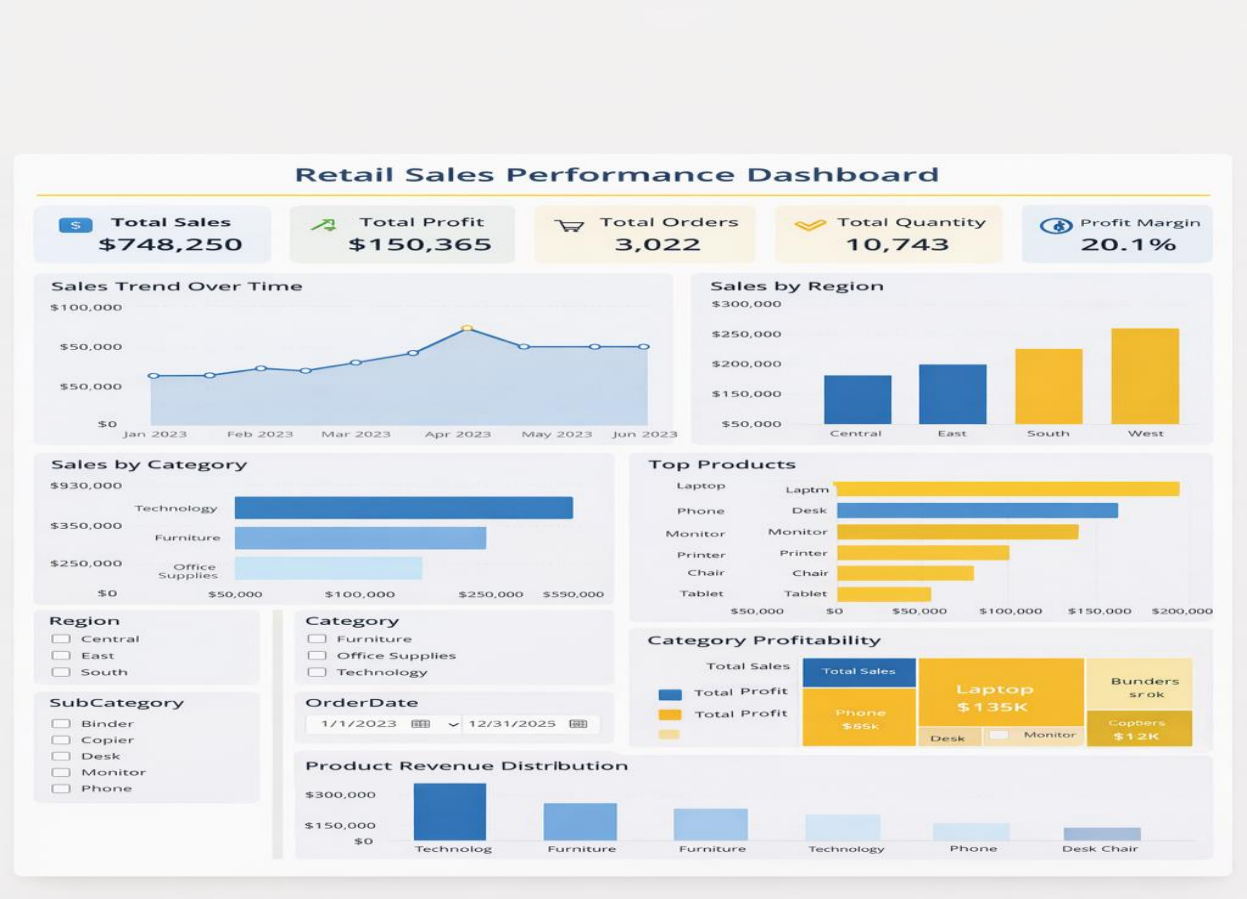
Add **Slicers**:

```
Region
Category
Product_Name
Order_Date
```

Users can filter the entire dashboard dynamically.

Final Dashboard Layout

```
-----------------------------------------------
         RETAIL SALES PERFORMANCE DASHBOARD
-----------------------------------------------
KPI CARDS
Sales | Profit | Orders | Margin | Quantity
-----------------------------------------------
Sales Trend | Sales by Region
-----------------------------------------------
Sales by Category | Top Products
-----------------------------------------------
Category Profitability | Product Treemap
-----------------------------------------------
Filters (Region | Category | Date)
```



12-03-2026

Problem Statement

A company wants to analyze its **supply chain and inventory performance** to improve operational efficiency.

The management wants to answer these questions:

- Which **products have the highest stock levels**
- Which **suppliers provide the most products**
- Which **warehouses store the most inventory**
- What is the **inventory value across the supply chain**
- Which products may lead to **stockouts or overstock**

Build an **interactive Power BI dashboard** that helps managers monitor inventory and make better logistics decisions.

Dataset Fields

Your supply chain dataset usually contains columns like:

```
Product_ID
Product_Name
Category
Supplier
Warehouse
Stock_Quantity
Unit_Price
Lead_Time
Order_Date
```

Step 1 — Import Dataset

Open **Microsoft Power BI Desktop**

1. Click **Get Data**
2. Select **CSV**
3. Load the dataset.

Step 2 — Create Important Measures (KPIs)

Go to **Modeling → New Measure**

Total Inventory

```
Total Inventory = SUM(Inventory[Stock_Quantity])
```

Inventory Value

```
Inventory Value =
SUMX(Inventory,
Inventory[Stock_Quantity] * Inventory[Unit_Price])
```

Average Lead Time

```
Avg Lead Time = AVERAGE(Inventory[Lead_Time])
```

Total Products

```
Total Products = DISTINCTCOUNT(Inventory[Product_ID])
```

Step 3 — KPI Cards (Top Section)

Create **Card visuals**.

Add:

- Total Inventory
- Inventory Value
- Average Lead Time
- Total Products

Example layout:

```
-----------------------------------------------
Total Inventory | Inventory Value | Avg Lead Time | Products
-----------------------------------------------
```

Step 4 — Inventory by Warehouse

Add **Bar Chart**

Fields:

```
Axis → Warehouse
Values → Total Inventory
```

Insight:

Shows which warehouse stores the most stock.

Step 5 — Inventory by Product Category

Add **Column Chart**

```
Axis → Category
Values → Total Inventory
```

This helps identify **high-demand product categories**.

Step 6 — Supplier Performance

Add **Bar Chart**

```
Axis → Supplier
Values → Total Inventory
```

Insight:

Identify suppliers contributing the most inventory.

Step 7 — Inventory Value by Product

Add **Treemap**

```
Group → Product Name
Values → Inventory Value
```

This shows **which products have the highest inventory value**.

Step 8 — Lead Time Analysis

Add **Column Chart**

```
Axis → Supplier
Values → Avg Lead Time
```

Insight:

Identify suppliers with slow delivery times.

Step 9 — Inventory Trend Over Time

Add **Line Chart**

```
X-axis → Order_Date
Values → Total Inventory
```

This shows **inventory growth or reduction trends**.

Step 10 — Add Filters

Add **Slicers**:

```
Category
Supplier
Warehouse
```

```
Order Date
```

This allows users to **interactively explore the data**.

Final Dashboard Layout

```
-----------------------------------------------
 SUPPLY CHAIN ANALYSIS DASHBOARD
-----------------------------------------------
 KPI CARDS
 Inventory | Inventory Value | Lead Time | Products
-----------------------------------------------
 Inventory by Warehouse | Inventory by Category
-----------------------------------------------
 Supplier Performance | Inventory Trend
-----------------------------------------------
 Product Value Treemap
-----------------------------------------------
 Filters (Supplier | Warehouse | Category)
```

13-03-2026

Problem Statement

A company wants to analyze its **financial performance across departments** to understand:

- Which departments generate the most **revenue**
- Which departments have the highest **expenses**
- Which departments contribute the most **profit**
- How **revenue and profit change over time**

The goal is to build an **interactive Power BI dashboard** that helps management make **data-driven financial decisions**.

Dataset Fields

```
Date
Department
Revenue
Expenses
Profit
```

Step 1 — Import Dataset

Open **Microsoft Power BI Desktop**

1. Click **Get Data**
2. Select **CSV**
3. Load your dataset.

Step 2 — Create DAX Measures (KPIs)

Go to **Modeling → New Measure**

Total Revenue

```
Total Revenue = SUM(Financial[Revenue])
```

Total Expenses

```
Total Expenses = SUM(Financial[Expenses])
```

Total Profit

```
Total Profit = SUM(Financial[Profit])
```

Profit Margin

```
Profit Margin = DIVIDE([Total Profit],[Total Revenue])
```

Step 3 — KPI Cards (Top Section)

Create **Card visuals**.

Add these KPIs:

- Total Revenue
- Total Expenses
- Total Profit
- Profit Margin

Example layout:

```
-------------------------------------------------
Total Revenue | Total Expenses | Total Profit | Profit Margin
-------------------------------------------------
```

This is the **KPI section like the dashboard image you showed earlier**.

Step 4 — Revenue Trend Over Time

Add **Line Chart**

Fields:

```
X-Axis → Date
Values → Total Revenue
```

Insight:

Shows how revenue changes across time.

Step 5 — Profit Trend

Add **Line Chart**

```
X-Axis → Date
Values → Total Profit
```

This shows **financial growth or loss patterns**.

Step 6 — Revenue by Department

Add **Bar Chart**

```
Axis → Department
Values → Total Revenue
```

Insight:

Identify which department generates the most revenue.

Step 7 — Expenses by Department

Add **Bar Chart**

```
Axis → Department
Values → Total Expenses
```

This helps detect **high spending departments**.

Step 8 — Profit by Department

Add **Column Chart**

```
Axis → Department
Values → Total Profit
```

This shows **which departments are most profitable**.

Step 9 — Revenue vs Expenses Comparison

Add **Clustered Column Chart**

```
Axis → Department
Values → Revenue and Expenses
```

Insight:

Compare spending vs income.

Step 10 — Department Contribution (Treemap)

Add **Treemap**

```
Group → Department
Values → Total Profit
```

This visual shows **department profit contribution**.

Step 11 — Add Filters

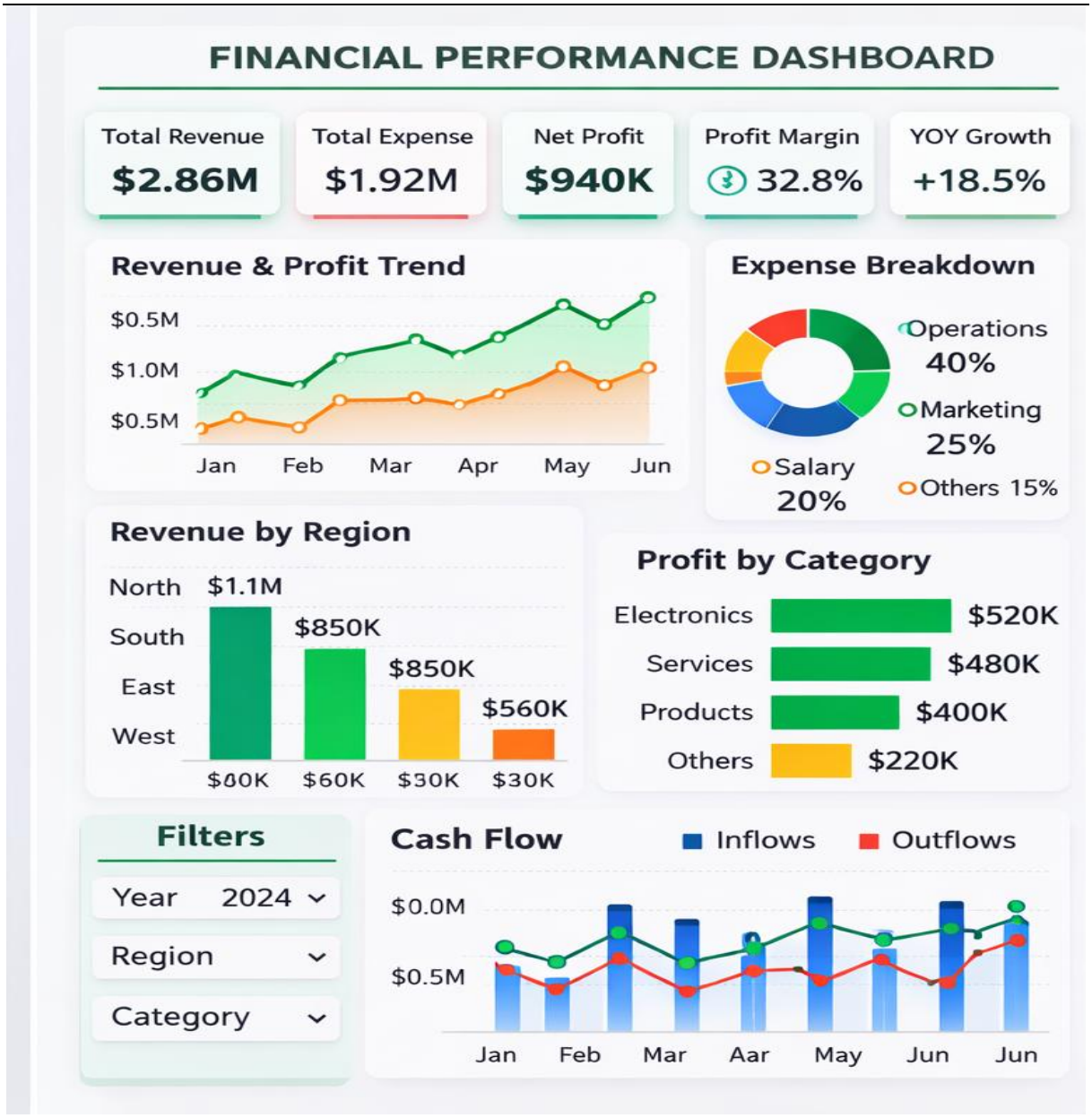
Add **Slicer**

```
Date
Department
```

Users can filter the dashboard dynamically.

Final Dashboard Layout

```
-----------------------------------------------
 FINANCIAL ANALYSIS DASHBOARD
-----------------------------------------------
 KPI CARDS
 Revenue | Expenses | Profit | Profit Margin
-----------------------------------------------
 Revenue Trend | Profit Trend
-----------------------------------------------
 Revenue by Department | Expenses by Department
-----------------------------------------------
 Revenue vs Expenses | Profit Treemap
-----------------------------------------------
 Filters (Date, Department)
```



14-03-2026

Dataset columns are:

```
PatientID
Disease
AdmissionDate
DischargeDate
Doctor
TreatmentCost
```

1. Problem Statement

Hospitals need to analyze **patient admissions, treatment costs, and disease distribution** to improve healthcare services and manage resources effectively.

Hospital management wants to answer the following questions:

- Which **diseases are most common among patients**
- Which **doctors treat the most patients**
- What is the **total treatment cost incurred by the hospital**
- How long patients stay in the hospital
- Which diseases generate the **highest treatment costs**

The objective is to build an **interactive healthcare analytics dashboard** to monitor hospital performance, patient trends, and financial impact.

2. Business Objectives

The dashboard should help hospital administrators:

- Monitor **total patient admissions**
- Analyze **treatment cost trends**
- Identify **high-cost diseases**
- Track **doctor workload**
- Evaluate **average hospital stay duration**

3. Key KPIs (Cards in Dashboard)

Create these measures in Power BI.

Total Patients

```
Total Patients = DISTINCTCOUNT(HospitalData[PatientID])
```

Total Treatment Cost

```
Total Cost = SUM(HospitalData[TreatmentCost])
```

Average Treatment Cost

```
Avg Treatment Cost = AVERAGE(HospitalData[TreatmentCost])
```

Total Doctors

```
Total Doctors = DISTINCTCOUNT(HospitalData[Doctor])
```

Average Hospital Stay

```
Avg Stay Days =
AVERAGEX(
HospitalData,
DATEDIFF(HospitalData[AdmissionDate],HospitalData[DischargeDate],DAY)
)
```

4. Dashboard Visuals

Patient Admissions Trend

Line Chart

```
X-Axis → AdmissionDate
Values → Total Patients
```

Insight: Shows **hospital admission trends over time**.

5. Disease Distribution

Bar Chart

```
Axis → Disease
Values → Total Patients
```

Insight: Identifies **most common diseases**.

6. Treatment Cost by Disease

Column Chart

```
Axis → Disease
Values → Total Cost
```

Insight: Shows **which diseases cost the hospital the most**.

7. Doctor Workload

Bar Chart

```
Axis → Doctor
Values → Total Patients
```

Insight: Shows **which doctors handle the most patients**.

8. Treatment Cost Distribution

Treemap

```
Group → Disease
Values → Total Cost
```

Insight: Shows **cost contribution of each disease**.

9. Patient Table

Create a **Table visual**

Columns:

```
PatientID
Disease
Doctor
TreatmentCost
AdmissionDate
DischargeDate
```

This helps administrators **view patient-level records**.

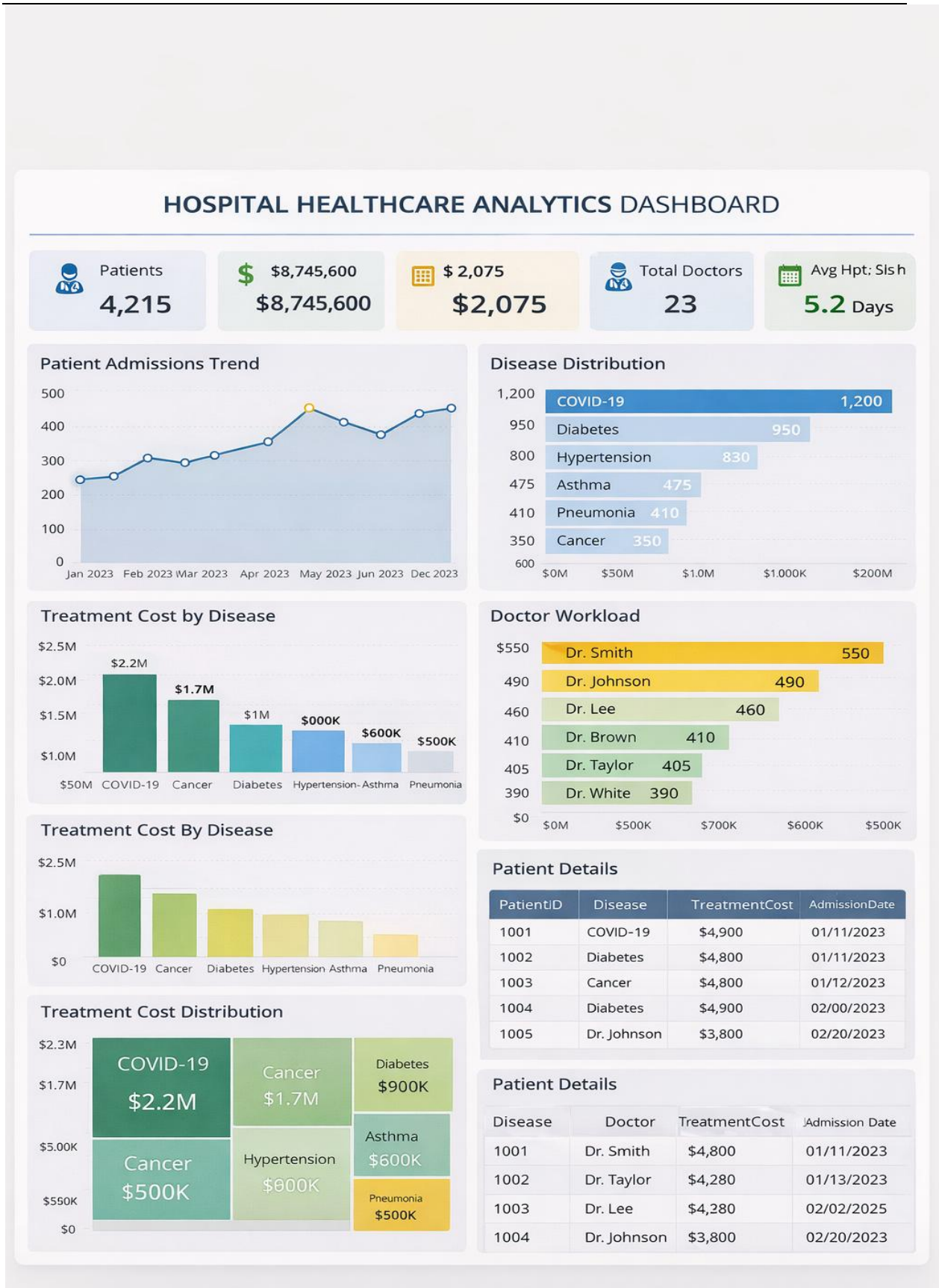
10. Filters (Slicers)

Add slicers for:

```
Disease
Doctor
AdmissionDate
```

This allows users to **filter hospital data interactively**.

11. Final Dashboard Layout

```
-----------------------------------------------
        HOSPITAL HEALTHCARE ANALYTICS DASHBOARD
-----------------------------------------------
KPI CARDS
Patients | Total Cost | Avg Cost | Doctors | Avg Stay
-----------------------------------------------
Patient Admissions Trend | Disease Distribution
-----------------------------------------------
Treatment Cost by Disease | Doctor Workload
-----------------------------------------------
Cost Treemap | Patient Details Table
```

Filters (Disease | Doctor | Date)